

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΑΡΧΕΙΩΝ

2η ΑΣΚΗΣΗ

Ον/μο: Θεόδωρος Κουτσαβδής

AM: 2017030145 (LAB20244094)

Για την δεύτερη εργαστηριακή άσκηση ήταν απαραίτητο να γίνουν ορισμένες μετρήσεις με όλες τις δομές δεδομένων που υλοποιήθηκαν. Οι μετρήσεις είναι:

1. Αριθμός συγκρίσεων ανά εισαγωγή.
2. Συνολικός χρόνος εισαγωγών.
3. Αριθμός συγκρίσεων ανά αναζήτηση (100 αριθμών).
4. Συνολικός χρόνος για 100 αναζητήσεις.
5. Αριθμός συγκρίσεων ανά αναζήτηση εύρους τιμών (100 αριθμών), για $K=100$ και $K=1000$.

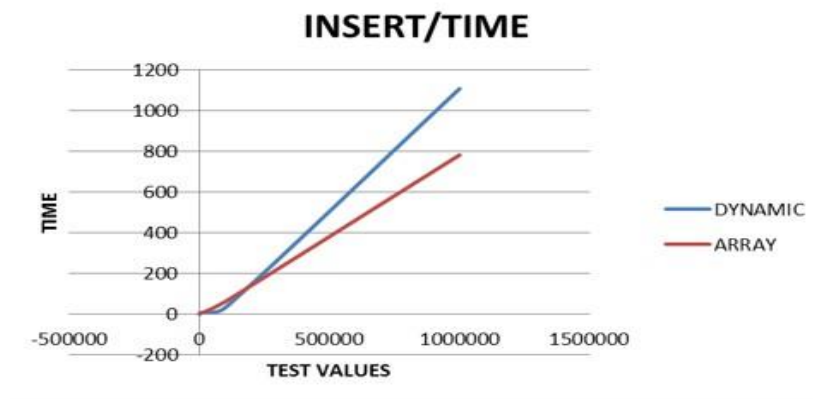
Παρακάτω είναι ένας ενδεικτικός πίνακας για τις τιμές που εμφανίζει το πρόγραμμα (Για $N=1000000$):

N = 1000000	AVG insert	Insert time	AVG search	Time search	AVG range K=100	AVG range K=1000
Dynamic BST	62	1180 ms	52	577500 ns	1312539	1294672
Array BST	103	861 ms	52	717600 ns	1312565	1294698
Sorted array	-	-	37	412700 ns	-	-

Ερμηνεία παραπάνω πίνακα

Το πρώτο πράγμα που παρατηρείται είναι ότι ο μέσος αριθμός συγκρίσεων ανά εισαγωγή του array BST είναι μεγαλύτερος από εκείνον του dynamic BST. Το γεγονός αυτό είναι λογικό καθώς ο κώδικας της insert() στην κλάση array BST έχει πιο πολλές συνθήκες σύγκρισης σε σχέση με τον αντίστοιχο κώδικα της κλάσης dynamic BST. Παρ'όλα αυτά παρατηρείται ότι ο χρόνος εισαγωγής είναι μικρότερος κατά την υλοποίηση array 3xN. Τέλος, αν η συνάρτηση insert της κλάσης array BST υλοποιηθεί με διαφορετικό τρόπο, τότε σίγουρα θα μειωθεί και ο μέσος αριθμός συγκρίσεων ανά αναζήτηση.

Παρακάτω παρουσιάζεται το διάγραμμα εισαγωγών ανά το χρόνο για όλα τα N (50,100, ...,1000000)



Όσον αφορά την υλοποίηση `search()` παρατηρείται ότι η δομή `binarySearch()` έχει το μικρότερο μέσο αριθμό συγκρίσεων ανά αναζήτηση, ενώ ταυτόχρονα οι άλλες δύο δομές έχουν ακριβώς τον ίδιο αριθμό (τα `if` statements της κάθε συνάρτησης επιβεβαιώνουν το γεγονός αυτό). Για το λόγο αυτό, όπως φαίνεται και στον παραπάνω πίνακα, η `binarySearch()` είναι η πιο γρήγορη σε σχέση με τις άλλες δύο δομές δεδομένων, ενώ η `array BST` είναι η πιο αργή. Κάτι τέτοιο ισχύει τις περισσότερες φορές και οι τιμές αυτές δεν είναι απόλυτες, καθώς το πρόγραμμα κάθε φορά αναζητά 100 διαφορετικούς τυχαίους αριθμούς.

Όλα τα παραπάνω σχετικά με την `search()` επιβεβαιώνονται από το παρακάτω διάγραμμα:

